

Mango Music CLI Tool

Workshop 7w

Version 7.1 Y

Project Description

MangoMusic is a command-line music streaming application backed by a MySQL database. The app is structured around the Data Access Object (DAO) pattern. Each entity in the system has a dedicated class responsible for all database interactions.

The application is already built and mostly running. Your job is not to build it from scratch. Your job is to work the way most developers work every day: inheriting an existing codebase, understanding it, fixing what is broken, and adding new functionality on top.

Application Overview

MangoMusic tracks artists, albums, users, and album play history. Staff use a console menu to search the catalog, look up users, view listening history, and generate business reports.

Architecture: The DAO Pattern

All database operations are organized into Data Access Objects. Each DAO handles one entity type and communicates with the database through a shared DataManager that manages connections.

DAO Class	Responsibility
ArtistDao	Search and retrieve artist records
AlbumDao	Album lookups, genre search, title search
UserDao	User lookup by username and email
AlbumPlayDao	User play history retrieval
ReportsDao	Analytics and business intelligence reports
DataManager	Manages the database connection for all DAOs

Database Configuration

Database connection settings live in `application.properties` at the project root. Update this file with your local MySQL credentials before starting.

```
db.username=root
```

```
db.password=yourpassword
```

Requirements

Setup Steps

1. Download the MangoMusic-CLI zip file from the course repository
2. Extract the zip to your `~/pluralsight/debugging-projects/` directory
3. Open IntelliJ IDEA and select File > Open, then navigate to the extracted MangoMusic-CLI folder
4. Wait for Maven to finish downloading dependencies
5. Import the `mango_music.sql` file into MySQL Workbench to create and seed the database
6. Open `application.properties` and update the username and password to match your local MySQL setup
7. Run `Main.java` and confirm the application launches to the home menu
8. Navigate through the menus to confirm there are no startup errors before beginning any tickets

Verifying Your Setup

When the application starts correctly you should see the following main menu. If you see a database connection error instead, double check your `application.properties` credentials and confirm MySQL is running.

Steps

Your Workflow

Work through tickets in priority order. Fix all bugs before starting features. Within bugs, tackle Critical priority first, then High, Medium, and Low.

Debugging Strategy

Follow this approach for every bug ticket:

1. Read the ticket completely before touching any code
2. Attempt to reproduce the bug, follow the steps and confirm you see the broken behavior
3. Identify the class and method named in the ticket
4. Read the error message carefully, `SQLException` messages tell you exactly what is wrong
5. Compare your Java code to your actual SQL schema in MySQL Workbench
6. Fix the issue using proper JDBC practices
7. Confirm the bug is fixed by reproducing the original steps and confirming the fix works
8. Run a manual SQL query to verify the data returned is correct

Feature Development Strategy

For feature tickets, write SQL before writing Java:

1. Write and test your SQL query directly in MySQL Workbench until results look correct
2. Note the exact column names your query returns, you will reference them in Java
3. Add the method to `ReportsDao.java` following the pattern of existing report methods
4. Use try-with-resources for all JDBC resources
5. Update `SpecialReportsScreen.java` to call your new DAO method and display results
6. Run the report in the app and compare output to your MySQL Workbench results

Progress Tracker

Done	Ticket	Method	Issue	Priority
☐	Bug #1	ArtistDao.searchArtists()	Code quality - missing resource cleanup	High
☐	Bug #2	AlbumDao.getAlbumsByArtist()	SQLException - column name mismatch	Critical
☐	Bug #3	UserDao.searchUsers()	PreparedStatement parameter binding error	Medium
☐	Bug #4	AlbumPlayDao.getUserRecentPlays()	Resource leak - connection not closed	High
☐	Bug #5	AlbumDao.getAlbumsByGenre()	SQL injection - Statement not PreparedStatement	Critical
☐	Bug #6	AlbumDao.searchAlbums()	Inefficient ResultSet getter	Low
☐	Feat #1	ReportsDao.getMostPlayedAlbumsByGenre()	Build from scratch	Medium
☐	Feat #2	ReportsDao.getUserDiversityReport()	Build from scratch	Medium
☐	Feat #3	ReportsDao.getPeakListeningHoursReport()	Build from scratch	Low

Bug Tickets

Six bugs have been intentionally introduced into the codebase. Each represents a real category of JDBC mistakes that developers will encounter on the job.

Bug #1: Artist Search - Missing Resource Cleanup

Priority: High Class: `ArtistDao` Method: `searchArtists()`

The artist search feature works and returns correct results, but the code is not following best practices for resource management.

Database resources are being opened without being properly closed afterward.

This is a code quality issue that will cause the application to slow down and eventually become unstable over extended use.

Refactor the method to handle resources correctly.

Bug #2: View Artist Albums Feature Not Working

Priority: Critical Class: `AlbumDao` Method: `getAlbumsByArtist()`

When trying to view albums for an artist, the application throws a `SQLException` and returns no results.

The feature is completely broken. Read the error message carefully.

Bug #3: User Search Only Finds Username Matches

Priority: Medium Class: `UserDao` Method: `searchUsers()`

The user lookup is supposed to search both username and email.

Searching 'gmail' should find users with gmail addresses.

Currently, only the username field is actually being searched and the email search silently returns wrong results.

Bug Tickets

Bug #4: View User Plays Causes Application Instability

Priority: High Class: `AlbumPlayDao` Method: `getUserRecentPlays()`

After viewing play history for 10-15 users, the application becomes slow and may show errors about too many connections.

This method manages its database resources differently than the other working methods and compare them carefully.

Bug #5: Find Albums by Genre - SQL Injection Vulnerability

Priority: Critical Class: `AlbumDao` Method: `getAlbumsByGenre()`

The security team has flagged this method as vulnerable to SQL injection.

Normal input works fine, but malicious input returns unexpected results.

The fix requires switching from one JDBC query approach to the safer alternative.

Bug #6: Search Albums Uses Inefficient `ResultSet` Getter

Priority: Low Class: `AlbumDao` Method: `searchAlbums()`

This feature works correctly but one column is being retrieved as the wrong type and then converted unnecessarily.

This is poor practice and could fail unexpectedly.

Use the `ResultSet` getter that matches the actual column data type.

Feature Tickets

Three new reports need to be built from scratch. For each feature you will write the SQL, add a method to `ReportsDao.java`, and update `SpecialReportsScreen.java` to display results.

Feature #1: Most Played Albums by Genre

Priority: Medium Method: `ReportsDao.getMostPlayedAlbumsByGenre()`

The content acquisition team wants to see the top 5 most-played albums within each genre.

Required output columns: `genre`, `album_title`, `artist_name`, `play_count`, `genre_rank`.

Include only genres with at least 5 albums that have plays.

Order by `genre`, then `play_count` descending.

Hint: Use a correlated subquery to calculate rank by counting how many albums in the same genre have more plays than the current album, then add 1. Wrap everything in an outer query and filter `WHERE genre_rank <= 5`.

Feature #2: User Listening Diversity Score

Priority: Medium Method: `ReportsDao.getUserDiversityReport()`

The personalization team wants to segment users into explorers versus specialists.

Calculate a diversity score for each user with 20 or more plays using: $(\text{distinct_genres} / \text{distinct_artists}) * 100$.

Show top 100 users ordered by `diversity_score` descending.

Required output columns: `user_id`, `username`, `subscription_type`, `distinct_genres_played`, `distinct_artists_played`, `total_plays`, `diversity_score`.

Hint: Use `COUNT(DISTINCT ...)` for both genres and artists. Write the formula as `(genre_count * 100.0 / artist_count)` and use `100.0` not `100` to avoid integer division. Use a `HAVING` clause to enforce the 20+ plays minimum.

Feature Tickets

Feature #3: Peak Listening Hours Analysis

Priority: Low Method: `ReportsDao.getPeakListeningHoursReport()`

The product team wants to see listening activity broken down by hour of day (0-23) to optimize release timing and server capacity.

Required output columns: `hour_of_day`, `total_plays`, `unique_users`, `avg_plays_per_user`.

Include all 24 hours ordered by `hour_of_day` ascending.

Hint: Use `HOUR(played_at)` to extract the hour from the `played_at` timestamp. Group by this extracted value. Calculate `avg_plays_per_user` as `COUNT(play_id) / COUNT(DISTINCT user_id)` and wrap in `ROUND(..., 2)`.

Bonus Tickets

These tickets are for students who finish the core features early. They involve more complex SQL patterns and are intentionally less guided. Use the working DAO methods as reference.

Bonus Feature #1: Album Completion Leaderboard

Difficulty: Hard Priority: Medium Method:

`ReportsDao.getAlbumCompletionLeaderboard()`

The content team wants to know which albums people actually finish versus which ones they abandon.

Show the top 25 albums ranked by completion rate.

Only include albums with at least 50 total plays.

Required output columns: `album_title`, `artist_name`, `genre`, `total_plays`, `completed_plays`, `completion_rate`.

Order by `completion_rate` descending.

Hint: Use `SUM(CASE WHEN ap.completed = TRUE THEN 1 ELSE 0 END)` to count completed plays. Use a `HAVING` clause (not `WHERE`) to enforce the 50-play minimum and `WHERE` runs before grouping and won't have access to the aggregated count. Use 100.0 not 100 in the percentage calculation to avoid integer division.

Bonus Tickets

Bonus Feature #2: Catalog Age vs. Engagement Analysis

Difficulty: Medium-Hard Priority: Low Method:

`ReportsDao.getCatalogAgeReport()`

The acquisition team wants to know whether listeners prefer recent releases or classic catalog titles.

Group all albums into four eras based on `release_year` and show engagement metrics per era.

Required output columns: `era`, `album_count`, `total_plays`, `avg_plays_per_album`, `completion_rate`. Era labels: Recent (Last 5 Years), Modern (6-10 Years Ago), Classic (11-20 Years Ago), Vintage (20+ Years Ago). Order by `avg_plays_per_album` descending.

Hint: Use a CASE expression on `al.release_year` with `YEAR(CURDATE())` for the current year comparison. In MySQL you can `GROUP BY era` using the alias from your CASE expression in the `SELECT`. You will use two separate CASE expressions in the same query and one to label the era, one inside `SUM()` to count completions.

Bonus Feature #3: Top 3 Genres Per Country

Difficulty: Very Hard Priority: Medium Method:

`ReportsDao.getTopGenresByCountryReport()`

The international markets team wants to see the top 3 genres by play volume in every country.

This is a top-N-per-group problem: instead of top-5-per-genre, it is top-3-per-country.

Required output columns: `country`, `genre`, `play_count`, `country_rank`.

Show only `country_rank <= 3`.

Order by `country` ascending, then `play_count` descending within each country.

In the UI, print a blank line when the country changes to visually group the output.

Hint: The structure is nearly identical to `getMostPlayedAlbumsByGenre()`, the rank for a (country, genre) row is: how many other genres in the same country have more plays than this one, plus 1. You need to join the users table to get country in both the outer query and inside the correlated subquery. The correlated subquery must match on country to stay within the same country.

What Makes a Good Project

Resource Management

- * All `Connection`, `PreparedStatement`, and `ResultSet` objects are closed after use
- * `try-with-resources` is used for automatic closing - not manual `try/catch/finally`
- * No database connections left open after a method returns
- * Resources are closed even when exceptions occur

Query Safety

- * `PreparedStatement` is used for all queries that accept user input
- * No string concatenation used to build SQL queries with user data
- * All `?` placeholders have corresponding `setXxx()` calls
- * Parameter indices are correct (1-based, one per placeholder)

ResultSet Usage

- * Column names in `getString()`, `getInt()` etc. match actual database column names exactly
- * The getter method matches the column's actual data type
- * No unnecessary type conversions such as `Integer.parseInt(rs.getString(...))`

Code Quality

- * New DAO methods follow the same pattern as existing working methods
- * SQL queries are tested in MySQL Workbench before being added to Java
- * Error messages are descriptive and include the method name
- * Results are correctly mapped to model objects or `ReportResult` instances

Testing Your Work

For every ticket, verify your fix or feature against the database directly. The feature tickets include verification queries. Run them in MySQL Workbench and compare the numbers to what your application displays. If they match, you are done.

When you see a `SQLException`, read the full message before touching any code. The message will tell you the table name if a column is missing, the column name if the type is wrong, or the line number if the syntax is invalid. Most JDBC bugs are diagnosed by reading the error, not by guessing.